

CREATIONAL DESIGN PATTERN

Singleton Design Pattern

Md Saeed Siddik

Assistant Professor,
Institute of IT, University of Dhaka

1

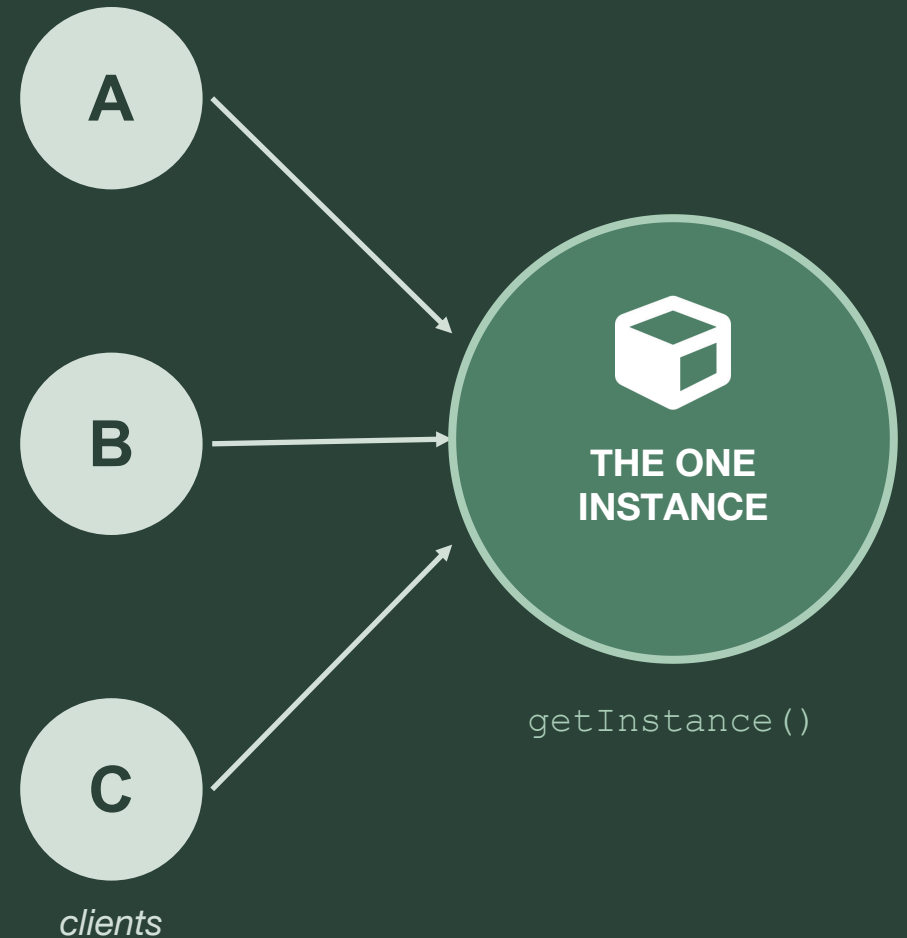
instance

5

Java implementations

4

ways it can break



What we will cover

Concept → problem → refactoring → hardening



01

The concept

Intent, analogy, structure and the implementation checklist



02

Real-world cases

Where Singleton is used, and the three criteria for using it



03

A problematic case

A Java app where every service opens its own connection



04

Refactoring

The step-by-step recipe, then a fix for the race condition



05

Five implementations

Eager, synchronized, double-checked, holder and enum



06

Hardening & judgment

Reflection, serialization, testing and when to avoid it

What is the Singleton pattern?

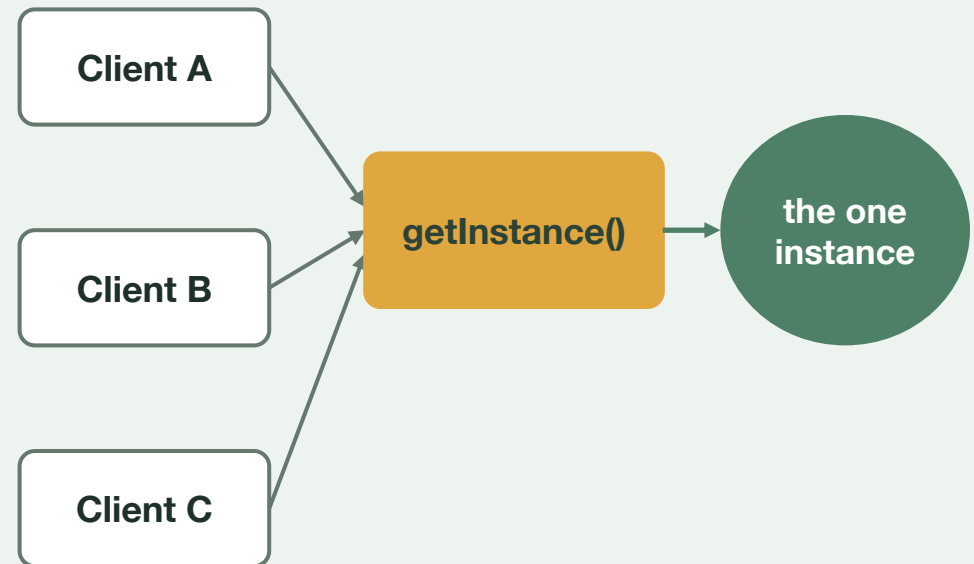
Ensure a class has only one instance, and provide a global point of access to it.



ANALOGY

A country has one official government. Whoever holds office, “The Government of X” is the single, global point of access.

Also known as: “initialization on first use”, the instance is created just in time, when first requested.



Every client receives the same object, often without realizing it.

Two problems, solved at once

1

Exactly one instance

Control access to a shared resource such as a database or a file. A regular constructor always returns a new object, so a different mechanism is needed.

2

One global access point

Like a global variable, the instance is reachable from anywhere, but unlike a global variable, no other code can overwrite it.



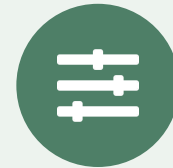
Because one class handles both jobs, Singleton bends the Single Responsibility Principle, a trade-off we will revisit at the end.

Where Singleton shows up



Logging systems

One consistent logging mechanism: every message flows through the same object.



Configuration managers

One source of truth for settings, read once and shared by the whole application.



Database connections

Opening connections is expensive; a single managed access point avoids waste.



Thread pools

A single pool of worker threads coordinates concurrent tasks centrally.

In the JDK: `Runtime.getRuntime()` always returns the one `Runtime` object of the running application.

Should you use it? Three criteria

Consider Singleton only if ALL three are true



Ownership is unclear

Ownership of the single instance cannot reasonably be assigned to one particular object.



Lazy initialization helps

Creating the instance on first use (not at start-up) is desirable.



No other global access

Global access to the instance is not already provided in another way.

ALL THREE TRUE

Singleton is a good fit.

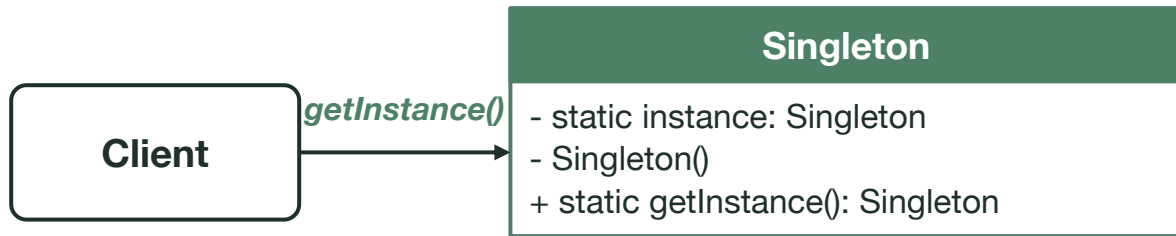
One shared instance with lazy creation and a global access point.

OTHERWISE

Pass the object as a parameter.

Simpler and more flexible than reaching for a global.

Structure and checklist



The class creates its instance on the first call and returns that same object every time afterwards.

Private constructor + **private static** instance + **public static** accessor. Nothing else is needed.

IMPLEMENTATION CHECKLIST

- 1 A private static attribute holds the single instance.
- 2 A public static accessor returns it.
- 3 The accessor does lazy initialization (creation on first use).
- 4 All constructors are private (or protected).
- 5 Clients use only the accessor, never new.

01

The problematic case

A small shop application where every service opens its own database connection

DatabaseConnection

OrderService

ReportService

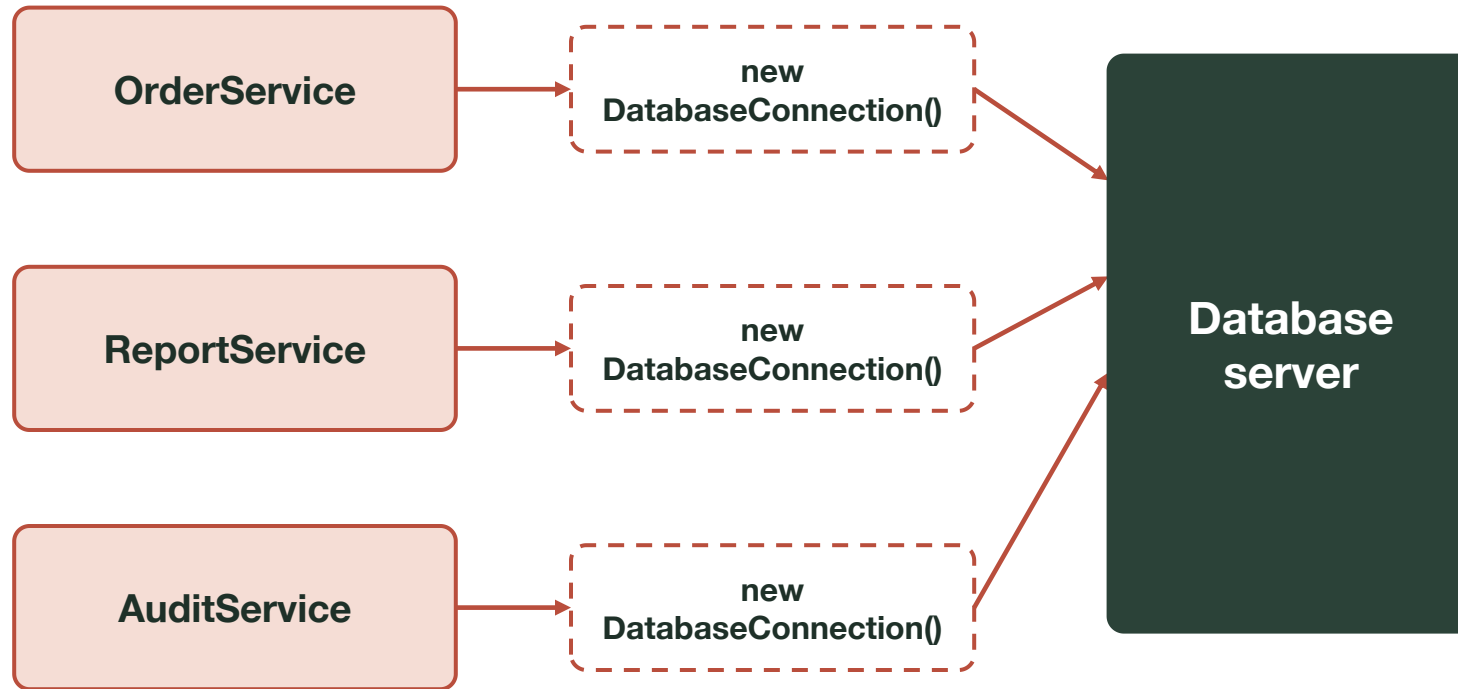
Java



PROBLEMATIC CASE

Every service opens its own connection

A shop application: several services, one database



3 services = 3 connections held open

WHAT GOES WRONG



Wasted resources

Each connection costs memory, sockets and authentication time.



Inconsistent state

Every copy keeps its own settings, cache and transactions.



No central control

Nothing caps, shares or throttles how many exist.

The problematic code (full listing)

DatabaseConnection.java

```
public class DatabaseConnection {
    private static int opened = 0;

    public DatabaseConnection() { // public!
        opened++;
        System.out.println("Opening DB connection...");
    }

    public void query(String sql) {
        System.out.println("Running: " + sql);
    }

    public static int getOpened() { return opened; }
}
```

ProblemDemo.java

```
class OrderService {
    DatabaseConnection db = new DatabaseConnection();
    void placeOrder() { db.query("INSERT ..."); }
}

class ReportService {
    DatabaseConnection db = new DatabaseConnection();
    void monthly() { db.query("SELECT ..."); }
}

public class ProblemDemo {
    public static void main(String[] args) {
        new OrderService().placeOrder();
        new ReportService().monthly();
        System.out.println("Connections opened: "
            + DatabaseConnection.getOpened());
    }
}
```

Public constructor: every new opens another expensive connection, and nothing can stop it.

Two services → two new → two connections

Run it: what actually happens

```
● ● ●  
$ java ProblemDemo  
Opening DB connection...  
Running: INSERT ...  
Opening DB connection...  
Running: SELECT ...  
Connections opened: 2
```

CONNECTIONS OPENED

2

expected

1

One shared connection is all this application needs.

Multiply by every service, every request and every thread, this is exactly the situation the Singleton pattern was designed for.

02

Refactoring to Singleton

Apply the recipe, discover the race condition, and fix it properly

Recipe

Race condition

Holder idiom

Verification



The refactoring recipe

Five steps, applied to DatabaseConnection

1 Add a private static field for the instance

```
private static DatabaseConnection instance;
```

2 Declare a public static creation method

```
public static DatabaseConnection getInstance()
```

3 Do lazy initialization inside that method

```
if (instance == null)
    instance = new DatabaseConnection();
```

4 Make the constructor private

```
private DatabaseConnection() { ... }
```

5 Replace every new in the client code

```
db = DatabaseConnection.getInstance(); // was: new
```

Result of the recipe: a lazy Singleton

```

DatabaseConnection.java (lazy version)

public class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() { } // private!

    public static DatabaseConnection getInstance() {
        if (instance == null) { // lazy
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public void query(String sql) { ... }
}

```

```

DatabaseConnection db = DatabaseConnection.getInstance();

```

Clients now call the accessor instead of new.

CHECKLIST

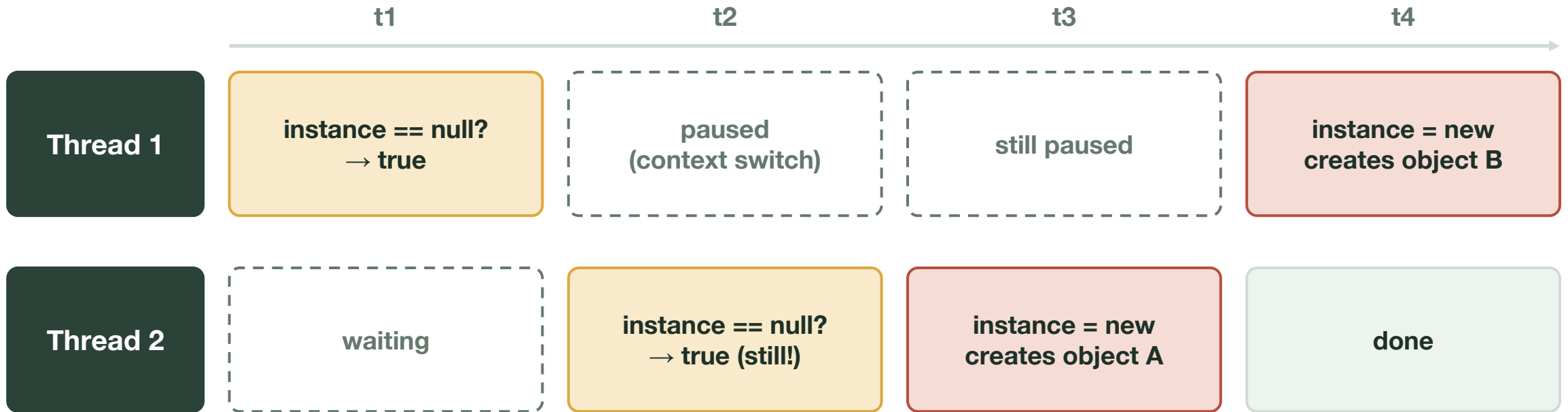
- ✓ Private static field
- ✓ Public static accessor
- ✓ Lazy initialization
- ✓ Private constructor
- ✓ Clients use the accessor



But what if two threads call `getInstance()` at the same moment?

Two threads, two instances

The check and the creation are separate steps



Both threads saw null, so both created an object: A and B. The Singleton guarantee is broken, and Thread 1 never re-checked after waking up.

Reproducing the race in Java

RaceDemo.java

```
import java.util.Set;
import java.util.concurrent.*;

public class RaceDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(5);
        CountDownLatch start = new CountDownLatch(1);
        Set<Object> seen = ConcurrentHashMap.newKeySet();

        for (int i = 0; i < 5; i++) {
            pool.submit(() -> {
                start.await(); // release all threads together
                seen.add(DatabaseConnection.getInstance());
                return null;
            });
        }
        start.countDown();
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("Distinct instances: " + seen.size());
    }
}
```



\$ java RaceDemo

Opening connection #3
Opening connection #1
Opening connection #2
Opening connection #4
Opening connection #5

Distinct instances: 5

Expected: 1 · Actual: 5

Five threads each built their own object. Order and count vary from run to run.

Refactored: a thread-safe Singleton

DatabaseConnection.java

```
public class DatabaseConnection {
    private static int opened = 0;

    private DatabaseConnection() { // private!
        opened++;
        System.out.println("Opening DB connection...");
    }

    private static class Holder { // lazy
        static final DatabaseConnection INSTANCE =
            new DatabaseConnection();
    }

    public static DatabaseConnection getInstance() {
        return Holder.INSTANCE;
    }

    public void query(String sql) {
        System.out.println("Running: " + sql);
    }

    public static int getOpened() { return opened; }
}
```

RefactoredDemo.java

```
class OrderService {
    DatabaseConnection db =
        DatabaseConnection.getInstance(); // was: new
    void placeOrder() { db.query("INSERT ..."); }
}

class ReportService {
    DatabaseConnection db =
        DatabaseConnection.getInstance(); // was: new
    void monthly() { db.query("SELECT ..."); }
}

public class RefactoredDemo {
    public static void main(String[] args) {
        new OrderService().placeOrder();
        new ReportService().monthly();
        System.out.println("Connections opened: "
            + DatabaseConnection.getOpened());
        System.out.println("Same instance? " + (
            DatabaseConnection.getInstance()
            == DatabaseConnection.getInstance()));
    }
}
```

Same demo, new result

```
$ java RefactoredDemo
Opening DB connection...
Running: INSERT ...
Running: SELECT ...
Connections opened: 1
Same instance? true
```

Why the holder idiom is safe

The JVM initializes a class only once, and the nested Holder class loads on the first call to `getInstance()`, lazy, with no locks.

CONNECTIONS OPENED

2 → 1

ProblemDemo → RefactoredDemo

INSTANCES MADE BY 50 RACING THREADS

50 → 1

lazy version → holder version

Five ways to write a Singleton in Java

Variant	Lazy?	Thread-safe?	Trade-off	Verdict
Eager (static final)	No, at class load	Yes (JVM)	Wastes memory if never used	Lightweight objects
Synchronized method	Yes	Yes	Lock on every call: slower	Simple; rarely-called code
Double-checked locking	Yes	Yes (needs volatile)	Fast, but easy to get wrong	When Holder is not an option
Holder (Bill Pugh)	Yes	Yes	No locks, no volatile	Standard Java choice
Enum	On first use	Yes	Also safe against reflection and serialization; cannot extend a class	Simplest, safest option

All five keep the constructor private and expose one accessor. They differ in WHEN the instance is created and HOW threads are protected.

Eager and synchronized

1 · Eager initialization

```
eager/Singleton.java
public class Singleton {
    private static final Singleton INSTANCE =
        new Singleton();

    private Singleton() { }

    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

- ✓ Thread-safe: the JVM initializes static fields once.
- ✓ Simple and fast to call.
- ✗ Created at class load, even if never used.

2 · Synchronized accessor

```
sync/Singleton.java
public class Singleton {
    private static Singleton instance;

    private Singleton() { }

    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

- ✓ Lazy and thread-safe.
- ✓ Easy to understand.
- ✗ Every call pays for the lock, even after creation.

3 · Double-checked locking

dcl/Singleton.java

```
public class Singleton {
    private static volatile Singleton instance;

    private Singleton() { }

    public static Singleton getInstance() {
        if (instance == null) { // check 1: no lock
            synchronized (Singleton.class) {
                if (instance == null) { // check 2: locked
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

WHY TWO CHECKS?

1

Check 1 (no lock)

After creation, callers skip the lock: the fast path.

2

Check 2 (inside lock)

Stops a second waiting thread from creating another object.

3

volatile

Prevents reordering, so no thread sees a half-built object.



Bill Pugh's work on this idiom led to Java 5 memory-model changes, and to the safer holder idiom on the next slide.

4 · Holder idiom (Bill Pugh)

```
holder/Singleton.java
public class Singleton {
    private Singleton() { }

    private static class Holder {
        static final Singleton INSTANCE =
            new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

WHY IT IS THE STANDARD

- ✓ **Lazy** The nested class is not loaded until `getInstance()` runs.
- ✓ **Thread-safe** The JVM runs class initialization exactly once.
- ✓ **Simple** No volatile, no synchronized, works in every Java version.

1 Application starts

Singleton is loaded. Holder is NOT loaded, no instance yet.

2 First `getInstance()` call

The method touches `Holder.INSTANCE` for the first time.

3 JVM loads Holder

`INSTANCE` is created exactly once and cached for ever.

5 · Enum singleton

```
enumtype/Singleton.java
public enum Singleton {
    INSTANCE;

    public void doSomething() {
        System.out.println("Doing something...");
    }
}

// usage
Singleton.INSTANCE.doSomething();
```

Limitation: an enum cannot extend another class (it may implement interfaces), and it is created on first use of the enum class.



Thread-safe

The JVM creates each enum constant exactly once.



Reflection-proof

Java refuses to create enum objects via reflection.

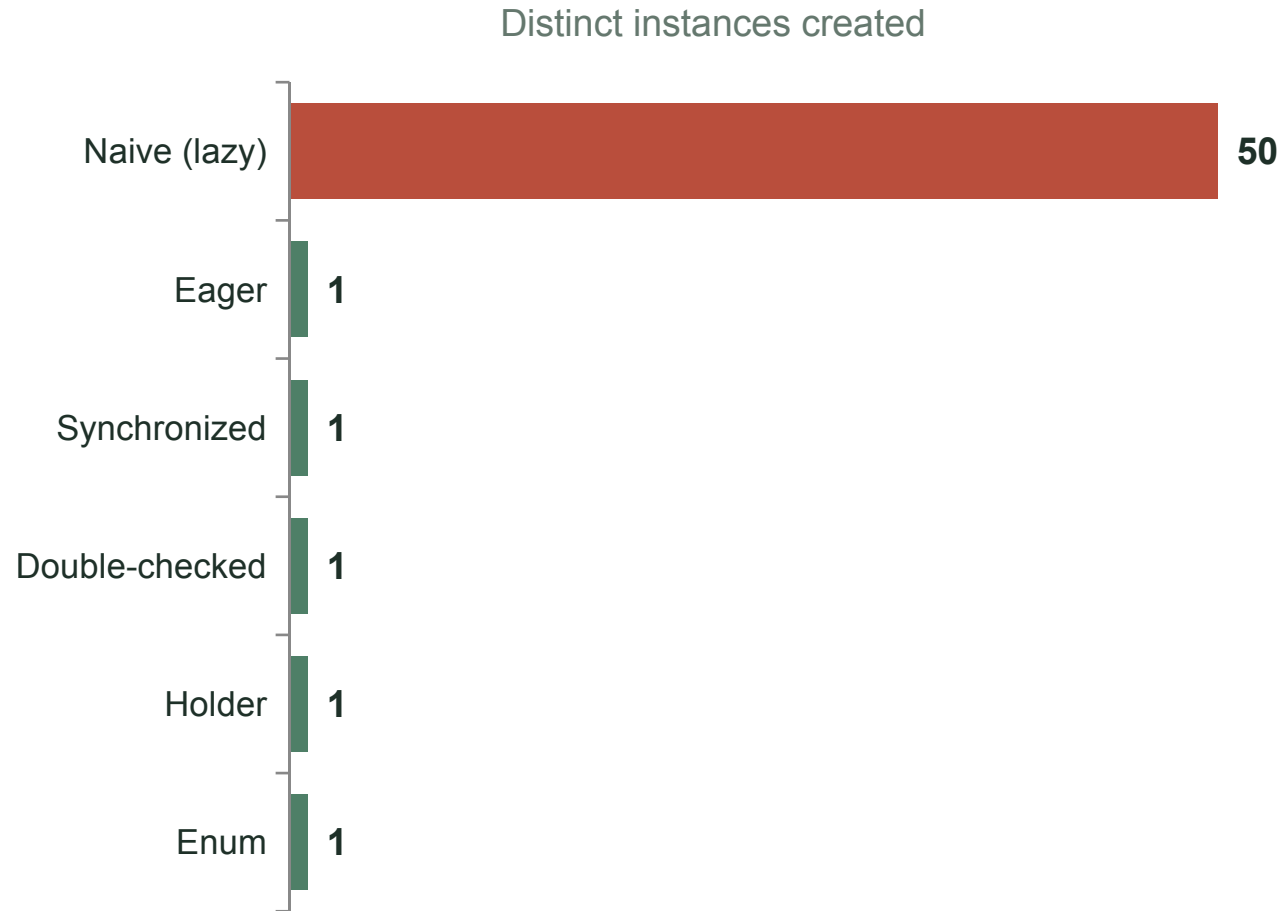


Serialization-safe

Deserializing an enum returns the same constant.

Often the simplest and safest choice when you do not need a base class.

Proof: 50 threads race for the instance



```
$ java VariantsDemo
Naive (lazy, unsafe): 50
Eager                : 1
Synchronized         : 1
Double-checked       : 1
Holder (Bill Pugh)   : 1
Enum                 : 1
```

Full code: 04_variants/VariantsDemo.java in the companion ZIP. Exact counts vary by run; the naive version always leaks extra instances.

03

Hardening & judgment

A Singleton can still be broken, hard to test, or simply unnecessary

Reflection

Serialization

Dependency injection

Trade-offs

Decision guide



Four ways a Singleton gets broken



Reflection

Attack `setAccessible(true)` opens the private constructor.

Defense Throw in the constructor if an instance exists, or use an enum.



Serialization

Attack Deserializing an object builds a brand-new instance.

Defense Add `readResolve()` that returns the instance, or use an enum.



Cloning

Attack A Cloneable Singleton can be copied with `clone()`.

Defense Do not implement Cloneable (or make `clone()` return the instance).



Several class loaders

Attack Each class loader can load its own copy of the class.

Defense Keep the class in one loader, or accept one instance per loader.

When the private constructor is not enough

ConfigManager.java

```
import java.io.Serializable;

public class ConfigManager implements Serializable {
    private static final ConfigManager INSTANCE =
        new ConfigManager();

    private ConfigManager() { }

    public static ConfigManager getInstance() {
        return INSTANCE;
    }
}
```

\$ java BreakDemo

Reflection: same object? false

Serialization: same object? false

BreakDemo.java

```
import java.io.*;
import java.lang.reflect.Constructor;

public class BreakDemo {
    public static void main(String[] args) throws Exception {
        ConfigManager one = ConfigManager.getInstance();
        // Attack 1: reflection
        Constructor<ConfigManager> c =
            ConfigManager.class.getDeclaredConstructor();
        c.setAccessible(true);
        System.out.println("Reflection: same object? "
            + (c.newInstance() == one));
        // Attack 2: serialization round-trip
        var bytes = new ByteArrayOutputStream();
        var out = new ObjectOutputStream(bytes);
        out.writeObject(one);
        out.close();
        var in = new ObjectInputStream(new ByteArrayInputStream(
            bytes.toByteArray()));
        Object copy = in.readObject();
        System.out.println("Serialization: same object? "
            + (copy == one));
    }
}
```

Hardened: guards, or just use an enum

```
ConfigManager.java (hardened)
import java.io.Serializable;

public class ConfigManager implements Serializable {
    private static final ConfigManager INSTANCE =
        new ConfigManager();

    private ConfigManager() {
        if (INSTANCE != null) { // reflection guard
            throw new IllegalStateException(
                "Already created!");
        }
    }

    public static ConfigManager getInstance() {
        return INSTANCE;
    }

    private Object readResolve() { // serialization guard
        return INSTANCE;
    }
}
```

SAME ATTACKS, HARDENED CLASS

```
● ● ●
$ java BreakDemo
Reflection: blocked -> Already created!
Serialization: same object? true
```

SAME ATTACKS, ENUM VERSION

```
● ● ●
$ java EnumBreakDemo
Reflection: blocked ->
    Cannot reflectively create enum objects
Serialization: same object? true
```

Guards: constructor check + readResolve(). An enum needs neither.

Singletons make code hard to test

Before: hidden dependency

```
OrderService.java
public class OrderService {
    public void placeOrder() {
        DatabaseConnection.getInstance() // hidden!
            .query("INSERT ...");
    }
}
```

```
● ● ●
$ java BeforeDemo
[real DB] INSERT ...
```

A “test” of OrderService writes to the real database, there is no way to swap in a fake.

After: dependency injection

```
Database.java + OrderService.java
public interface Database {
    void query(String sql);
}

public class OrderService {
    private final Database db; // an interface

    public OrderService(Database db) { // injected
        this.db = db;
    }

    public void placeOrder() {
        db.query("INSERT ...");
    }
}
```

The Singleton can stay, but only the application’s wiring code touches it.

Testing with a fake database

```
TestDemo.java
import java.util.*;

class FakeDatabase implements Database {
    final List<String> log = new ArrayList<>();
    public void query(String sql) { log.add(sql); }
}

public class TestDemo {
    public static void main(String[] args) {
        // Test: no real database needed
        FakeDatabase fake = new FakeDatabase();
        new OrderService(fake).placeOrder();
        boolean ok = fake.log.size() == 1
            && fake.log.get(0).startsWith("INSERT");
        System.out.println(ok ? "PASS: order was written" : "FAIL");

        // Production: wire in the real singleton once
        new OrderService(DatabaseConnection.getInstance())
            .placeOrder();
    }
}
```



```
$ java TestDemo
PASS: order was written
[real DB] INSERT ...
```



Fast No database needed to test the logic.



Repeatable The fake starts empty every time.



Swappable Production wires in the real Singleton once.

Advantages and limitations



Advantages

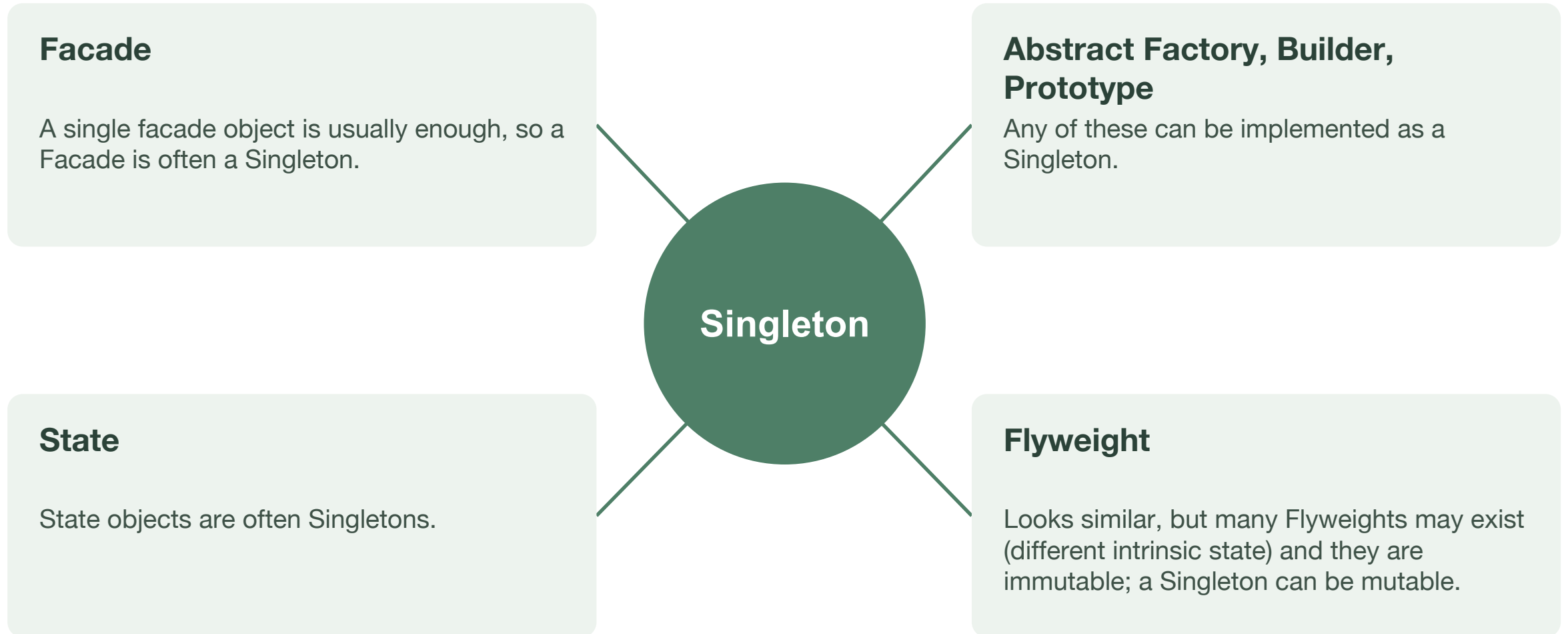
- ✓ Guarantees exactly one instance
- ✓ Global access point, protected from overwriting
- ✓ Lazy: created only when first requested
- ✓ Saves memory and connections
- ✓ Consistent shared state across the app



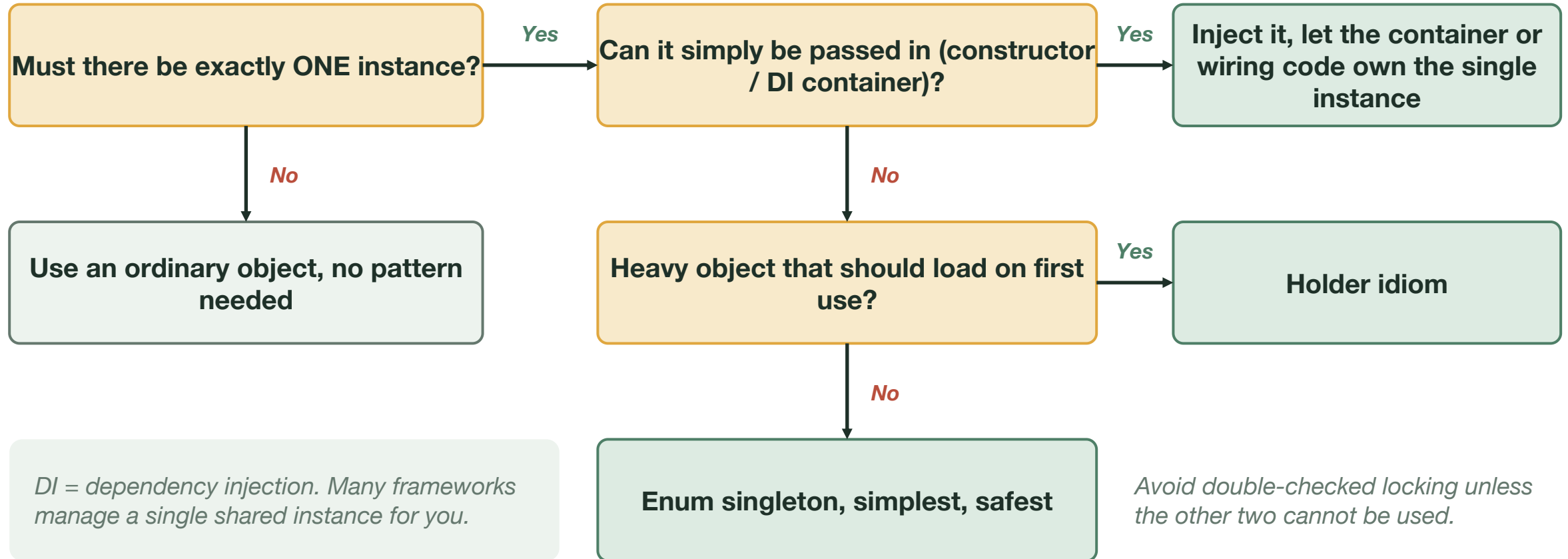
Limitations

- ✗ Violates the Single Responsibility Principle
- ✗ Can mask bad design and tight coupling
- ✗ Needs care in multithreaded code
- ✗ Hard to unit-test (private constructor, static access)
- ✗ A global in disguise: it renames the problem

Singleton and its neighbours



Decision guide: should it be a Singleton?



Class activity: build a ConfigManager

Work in pairs · 25 minutes · Java

- 1 Write it** ConfigManager loads a settings map once and exposes get(key). Make it a Singleton.
- 2 Race it** Start 50 threads that call getInstance(). Count the distinct instances.
- 3 Break it** Attack it with reflection and a serialization round-trip.
- 4 Harden it** Add the guards, or switch to an enum. Re-run your attacks.
- 5 Test it** Inject ConfigManager behind an interface and test a service with a fake.

DISCUSS

Which implementation did you pick, and why?

Could a plain object passed as a parameter have done the job?

What happens to your tests if two of them change the same Singleton?

Key takeaways



1

Singleton = one instance + one global access point: private constructor, private static field, public static accessor.

2

The classic lazy version fails under concurrency, we measured 50 instances from 50 threads.

3

In Java, prefer the holder idiom or an enum; avoid hand-rolled double-checked locking.

4

Guard against reflection and serialization, or let an enum do it for you.

5

A Singleton is a global in disguise: inject dependencies when you can.

Sources & companion code



SourceMaking

Intent, criteria, checklist and rules of thumb.

[Singleton pattern](#)



SourceMaking · Java

The holder idiom and Bill Pugh's background.

[Singleton in Java](#)



Refactoring.Guru

Problem, solution, how-to steps, pros and cons.

[Singleton](#)



GeeksforGeeks

Five implementations, thread safety, applications.

[Singleton design pattern](#)



COMPANION CODE (ZIP)

01_problem · 02_naive_race · 03_refactored
04_variants · 05_breaking · 06_testing
run_all.sh compiles and runs every example (JDK 11+)

NEXT CLASS

Factory Method, another way to control object creation.